

// Jivops

Seguridad en IA

Guía completa — 30 lecciones

Fundamentos · Ataques adversariales · Privacidad · Guardrails y defensa en profundidad · Supply chain · Operación y respuesta

Contenido

Módulo 1: Fundamentos de seguridad en IA

Lección 01 — Por qué la IA introduce una superficie de ataque distinta al software tradicional

Lección 02 — OWASP Top 10 para aplicaciones LLM

Lección 03 — Modelado de amenazas para sistemas de IA

Lección 04 — El ciclo de vida de ML y dónde puede introducirse cada riesgo

Lección 05 — Diferencia entre seguridad DEL modelo y seguridad CON el modelo

Módulo 2: Ataques adversariales y manipulación de entrada

Lección 06 — Prompt injection avanzado — directo vs indirecto

Lección 07 — Jailbreaking — técnicas y por qué funcionan

Lección 08 — Ataques adversariales sobre modelos de clasificación/visión

Lección 09 — Data poisoning — envenenamiento de datos de entrenamiento

Lección 10 — Ataques de evasión en producción

Módulo 3: Privacidad y extracción de información

Lección 11 — Model inversion — reconstruir datos de entrenamiento

Lección 12 — Membership inference — saber si un dato se usó para entrenar

Lección 13 — Model extraction — robo del modelo mediante consultas

Lección 14 — Fuga de PII en respuestas generadas

Lección 15 — Técnicas de privacidad diferencial en ML

Módulo 4: Guardrails y defensas en profundidad

Lección 16 — Arquitectura de defensa en capas para aplicaciones LLM

Lección 17 — Filtrado de entrada — sanitización de prompts

Lección 18 — Filtrado de salida — validación de respuestas antes de entregarlas

Lección 19 — Guardrails basados en clasificadores vs basados en reglas

Lección 20 — Rate limiting y throttling como defensa

Módulo 5: Supply chain y seguridad de infraestructura de IA

Lección 21 — Seguridad de la cadena de suministro de modelos (model supply chain)

Lección 22 — Riesgos de modelos y datasets de terceros no verificados

Lección 23 — Escaneo de vulnerabilidades en dependencias de ML

Lección 24 — Seguridad de API keys y credenciales de modelos

Lección 25 — Aislamiento y sandboxing de cargas de trabajo de inferencia

Módulo 6: Operación y respuesta

Lección 26 — Red teaming de sistemas de IA

Lección 27 — Detección de abuso y uso malicioso en producción

Lección 28 — Respuesta a incidentes de seguridad específicos de IA

Lección 29 — Cumplimiento normativo de seguridad de IA (EU AI Act, marcos regulatorios)

Lección 30 — Proyecto final: postura de seguridad de IA completa lista para producción

Módulo 1: Fundamentos de seguridad en IA

Lección 01 — Por qué la IA introduce una superficie de ataque distinta al software tradicional

El propio lenguaje natural de entrada se convierte en un vector de ataque potencial (como prompt injection), algo que no existe de la misma forma en aplicaciones tradicionales donde la entrada sigue formatos estructurados y validables más estrictamente.

Un campo estructurado como un número de teléfono tiene un formato predecible y reglas claras de validación (solo dígitos, cierta longitud); el lenguaje natural libre no tiene ese patrón fijo, así que una instrucción maliciosa disfrazada dentro de texto aparentemente normal es mucho más difícil de detectar y filtrar automáticamente que un valor que simplemente no cumple un formato esperado.

Lección 02 — OWASP Top 10 para aplicaciones LLM

Documenta de forma estandarizada los riesgos de seguridad más críticos y comunes específicos de aplicaciones basadas en LLM (como prompt injection o fuga de datos sensibles), dando un marco de referencia compartido para priorizar esfuerzos de seguridad.

Sin una terminología compartida, distintos equipos podrían describir el mismo tipo de riesgo (como prompt injection) con nombres o definiciones ligeramente distintas entre sí, dificultando la comunicación clara; un marco de referencia estandarizado y ampliamente reconocido como el OWASP Top 10 da un vocabulario común que facilita alinear prioridades de seguridad entre equipos distintos de la misma organización.

Lección 03 — Modelado de amenazas para sistemas de IA

Identificar sistemáticamente los activos valiosos del sistema (datos, modelo, credenciales), los posibles atacantes y sus motivaciones, y las vías de ataque específicas relevantes para ese sistema particular, para priorizar qué defensas implementar primero.

Aunque ambos sistemas usen la misma tecnología de LLM subyacente, las consecuencias de un error o un ataque exitoso son radicalmente distintas: un chatbot de atención al cliente con una respuesta incorrecta genera una mala experiencia, mientras un sistema médico con una respuesta incorrecta podría poner en riesgo real la salud de una persona, justificando un modelado de amenazas y un nivel de defensa mucho más riguroso en ese segundo caso.

Lección 04 — El ciclo de vida de ML y dónde puede introducirse cada riesgo

Un atacante podría introducir un riesgo en una etapa temprana (como envenenar los datos de entrenamiento) que comprometa el modelo de forma fundamental, un problema que ninguna protección aplicada solo en la etapa de inferencia final podría detectar o corregir después.

Si los datos de entrenamiento fueron manipulados deliberadamente antes de esa etapa (data poisoning), ese comportamiento malicioso quedaría incorporado dentro de los propios pesos aprendidos del modelo resultante; ninguna protección aplicada únicamente en la etapa final de inferencia podría detectar o corregir ese problema, ya que el modelo ya 'aprendió' ese comportamiento problemático desde su origen en el entrenamiento.

Lección 05 — Diferencia entre seguridad DEL modelo y seguridad CON el modelo

La seguridad DEL modelo se enfoca en defender el propio sistema de IA de ataques dirigidos hacia él (como prompt injection o robo del modelo), mientras la seguridad CON el modelo usa capacidades de IA para detectar o prevenir amenazas en otros sistemas distintos.

Aunque ese modelo esté cumpliendo la función valiosa de detectar amenazas en otros sistemas, sigue siendo en sí mismo un sistema de IA que un atacante sofisticado podría intentar manipular directamente (por ejemplo, con entradas diseñadas específicamente para evadir su detección); cumplir bien su función de seguridad CON el modelo no garantiza automáticamente que el modelo mismo esté protegido, siendo ambas dimensiones consideraciones independientes y necesarias.

Módulo 2: Ataques adversariales y manipulación de entrada

Lección 06 — Prompt injection avanzado — directo vs indirecto

En el ataque directo, el usuario malicioso escribe la instrucción manipuladora directamente en su propia entrada; en el indirecto, la instrucción maliciosa está incrustada en un contenido externo (como un documento o página web) que el sistema procesa como parte del contexto sin que el usuario final la haya escrito.

En un ataque directo, al menos hay alguien intentando conscientemente manipular el sistema; en un ataque indirecto, un usuario completamente legítimo e inocente podría hacer una pregunta normal que, al recuperar un documento contaminado con instrucciones ocultas, termine desencadenando un comportamiento malicioso sin que ni el usuario ni el sistema perciban de inmediato que algo salió mal.

Lección 07 — Jailbreaking — técnicas y por qué funcionan

Manipular al modelo mediante formulaciones creativas del prompt (como pedirle que actúe un rol ficticio, o presentar la solicitud de forma indirecta) para que ignore o eluda las restricciones de seguridad que normalmente aplicaría.

Al enmarcar la solicitud dentro de un contexto ficticio o de rol, se crea una capa de distancia aparente respecto a la solicitud real subyacente, lo que en ciertos casos puede hacer que el modelo procese esa petición de forma distinta a como procesaría la misma solicitud directa, potencialmente relajando las restricciones que normalmente aplicaría a un pedido sin ese marco narrativo adicional.

Lección 08 — Ataques adversariales sobre modelos de clasificación/visión

Una imagen modificada con perturbaciones sutiles (a menudo imperceptibles para un humano) diseñadas específicamente para hacer que el modelo la clasifique incorrectamente, aunque para un observador humano la imagen parezca completamente normal.

Si un atacante pudiera diseñar una perturbación sutil que engañe al sistema de reconocimiento facial para clasificar incorrectamente su identidad, podría potencialmente obtener acceso físico no autorizado a una instalación protegida; esa consecuencia práctica y real va mucho más allá de un simple error académico de clasificación en un contexto de laboratorio sin implicaciones directas.

Lección 09 — Data poisoning — envenenamiento de datos de entrenamiento

Manipular deliberadamente el comportamiento del modelo resultante (introduciendo un sesgo específico, una puerta trasera, o una vulnerabilidad) sin que el equipo que entrena el modelo se dé cuenta de que los datos fueron comprometidos.

Cualquier persona con acceso a contribuir contenido público en internet podría insertar deliberadamente datos maliciosos diseñados específicamente para ser recolectados por un scraper automático de entrenamiento; sin ninguna verificación adicional de esa fuente externa, esos datos contaminados terminarían mezclados con datos legítimos, mientras un dataset curado internamente con control sobre su origen tiene mucho menos exposición a ese tipo de manipulación externa deliberada.

Lección 10 — Ataques de evasión en producción

El ataque de evasión ocurre en tiempo de inferencia, contra un modelo ya entrenado y desplegado, manipulando la entrada específica de una consulta para evadir su comportamiento esperado, sin necesitar

comprometer el proceso de entrenamiento original.

Un ataque de evasión no depende de que los datos de entrenamiento hayan sido comprometidos, sino que explota directamente cómo el modelo ya entrenado (aunque completamente limpio en su origen) procesa una entrada específica diseñada deliberadamente para engañarlo en el momento de la consulta; la limpieza del entrenamiento original no garantiza inmunidad contra ese tipo distinto de manipulación aplicada después, en producción.

Módulo 3: Privacidad y extracción de información

Lección 11 — Model inversion — reconstruir datos de entrenamiento

Reconstruir aproximadamente información sobre los datos originales usados para entrenar el modelo, analizando cuidadosamente las respuestas del modelo ante distintas consultas diseñadas específicamente para ese propósito.

Si el ataque tuviera éxito, reconstruir información sobre pacientes reales a partir de un modelo médico representaría una violación grave y valiosa de privacidad, con consecuencias reales para esas personas; un modelo entrenado únicamente con datos públicos no sensibles no ofrecería ese mismo incentivo real, ya que la información que un atacante podría reconstruir ya sería pública de todas formas.

Lección 12 — Membership inference — saber si un dato se usó para entrenar

Si un registro de datos específico y conocido por el atacante fue o no parte del conjunto de datos usado para entrenar ese modelo particular, sin necesitar reconstruir el contenido completo de ese dato.

Si el dataset de entrenamiento fuera, por ejemplo, específicamente de pacientes con una condición médica particular, simplemente confirmar que el registro de una persona específica fue parte de ese conjunto ya revelaría que esa persona probablemente tiene esa condición, una información sensible y privada, sin necesitar reconstruir ningún detalle adicional del contenido completo de ese registro médico.

Lección 13 — Model extraction — robo del modelo mediante consultas

Realiza un gran número de consultas sistemáticas a la API pública del modelo, usando las respuestas obtenidas para entrenar un modelo propio que imite el comportamiento del modelo original, aproximándose a su funcionalidad sin haber accedido nunca a sus pesos reales.

Aunque el modelo extraído sea solo una aproximación (no una copia perfecta de los pesos internos), si esa aproximación captura suficientemente bien el comportamiento y la funcionalidad valiosa del modelo original, un competidor podría obtener un beneficio similar sin haber invertido los millones de dólares reales en investigación y entrenamiento que la empresa original sí invirtió, erosionando directamente esa ventaja competitiva.

Lección 14 — Fuga de PII en respuestas generadas

Si esa información personal específica aparecía repetidamente en los datos de entrenamiento (como en documentos públicos filtrados o mal protegidos), el modelo podría haber memorizado parcialmente ese patrón y reproducirlo cuando se le hace una consulta relacionada.

Una vez que el modelo memorizó cierta información durante el entrenamiento, esa memorización queda incorporada en sus pesos internos de forma difícil de revertir completamente sin reentrenar desde cero; un filtro aplicado únicamente después sobre las respuestas generadas tendría que anticipar y capturar cada posible forma en que esa información podría aparecer, una tarea mucho menos confiable que prevenir el problema desde el origen limpiando cuidadosamente los datos antes del entrenamiento.

Lección 15 — Técnicas de privacidad diferencial en ML

Añade ruido estadístico controlado durante el entrenamiento de forma que la presencia o ausencia de un único registro individual específico en el dataset tenga un impacto matemáticamente acotado y limitado en el resultado final del modelo, dificultando inferir información sobre ese individuo particular.

En un contexto donde el dataset contiene información sensible de personas reales (como registros médicos o financieros), proteger su privacidad individual mediante esa garantía matemática puede ser más importante que exprimir cada punto porcentual adicional de precisión del modelo; ese tradeoff aceptado deliberadamente refleja priorizar la protección real de las personas involucradas sobre el rendimiento técnico máximo posible del sistema.

Módulo 4: Guardrails y defensas en profundidad

Lección 16 — Arquitectura de defensa en capas para aplicaciones LLM

Combina múltiples capas de defensa independientes (filtrado de entrada, guardrails del modelo, filtrado de salida, monitoreo) de forma que si una capa específica falla o es evadida, las demás capas todavía ofrecen protección adicional.

Si el filtrado de entrada fuera la única defensa real del sistema, un atacante que descubra una forma específica de redactar su ataque que evada ese filtro particular tendría acceso directo y sin ninguna otra barrera hacia el resto del sistema; con múltiples capas independientes, esa misma evasión del filtrado de entrada todavía tendría que superar los guardrails del modelo y el filtrado de salida antes de tener éxito completo.

Lección 17 — Filtrado de entrada — sanitización de prompts

Patrones conocidos de intentos de manipulación (como instrucciones típicas de jailbreak o prompt injection), contenido claramente malicioso, o entradas que no cumplen con el formato o alcance esperado de la aplicación.

Un filtro de entrada típicamente se basa en reconocer patrones ya conocidos de ataques anteriores; un atacante suficientemente creativo y motivado podría diseñar deliberadamente una formulación completamente nueva que ese filtro nunca haya visto ni anticipado, evadiendo esa capa específica, razón por la cual el filtrado de entrada por sí solo nunca debería considerarse la única línea de defensa del sistema.

Lección 18 — Filtrado de salida — validación de respuestas antes de entregarlas

Detecta y bloquea contenido problemático que el modelo haya generado (como información sensible filtrada, contenido dañino, o una respuesta que no cumple el formato esperado), actuando como una última capa de verificación incluso si algo pasó las defensas anteriores.

Incluso si el filtrado de entrada y los guardrails del modelo funcionan correctamente en la gran mayoría de los casos, ninguna defensa es perfecta al 100%; el filtrado de salida actúa como una red de seguridad final que puede atrapar ese pequeño porcentaje de casos excepcionales donde, por alguna razón, una respuesta problemática logró pasar todas las capas anteriores sin ser detectada.

Lección 19 — Guardrails basados en clasificadores vs basados en reglas

Un guardrail basado en reglas fijas solo detecta coincidencias exactas predefinidas, mientras un clasificador puede generalizar y detectar contenido problemático con redacciones nuevas que nunca vio explícitamente antes, basándose en patrones aprendidos más amplios.

Una regla fija basada en coincidencia exacta de texto solo reconoce esa palabra específica predefinida, así que un sinónimo, una variación ortográfica deliberada, o una forma distinta de expresar la misma idea evadiría fácilmente esa detección literal; un clasificador entrenado para reconocer el significado o intención subyacente podría identificar ese contenido problemático independientemente de la redacción exacta específica usada.

Lección 20 — Rate limiting y throttling como defensa

Ataques de fuerza bruta que requieren muchos intentos repetidos (como probar múltiples variantes de un ataque de jailbreak), extracción masiva de información mediante consultas sistemáticas repetidas, y abuso general de recursos que generaría un costo desproporcionado.

Un ataque de model extraction exitoso depende de poder realizar un volumen alto de consultas en un tiempo razonablemente corto; si un rate limiting estricto obliga al atacante a esperar mucho más tiempo entre cada consulta permitida (o a usar muchas cuentas distintas para evadir ese límite), el ataque se vuelve significativamente más lento, costoso y además más fácil de detectar mientras está ocurriendo, gracias a ese patrón de comportamiento anómalo generado por el intento de evasión.

Módulo 5: Supply chain y seguridad de infraestructura de IA

Lección 21 — Seguridad de la cadena de suministro de modelos (model supply chain)

El modelo podría contener comportamiento malicioso deliberadamente incorporado (como una puerta trasera activada por ciertas entradas específicas), sin que el usuario que lo descarga tenga forma directa de saberlo con solo revisar los pesos numéricos del modelo.

El código fuente de una aplicación tradicional puede leerse línea por línea para entender exactamente qué hace, mientras un modelo de IA consiste en millones de parámetros numéricos sin ninguna representación legible directa; un comportamiento malicioso podría estar codificado sutilmente dentro de esos pesos sin ningún patrón obvio visible, haciendo que auditar visualmente un modelo en busca de esa intención maliciosa sea mucho más difícil que revisar código fuente tradicional legible.

Lección 22 — Riesgos de modelos y datasets de terceros no verificados

Evaluar la reputación y confiabilidad de la fuente, probar el modelo o dataset en un entorno aislado antes de integrarlo, y monitorear su comportamiento cuidadosamente incluso después de la integración inicial, en vez de confiar ciegamente sin ninguna verificación previa.

Si el modelo de tercero tuviera un comportamiento malicioso oculto, probarlo primero en un entorno aislado permitiría descubrir ese problema sin que haya afectado todavía a ningún usuario real o sistema de producción crítico; integrarlo directamente sin esa validación previa expondría inmediatamente a usuarios reales a un riesgo completamente no verificado desde el primer momento de uso.

Lección 23 — Escaneo de vulnerabilidades en dependencias de ML

Una librería con una vulnerabilidad conocida podría ser explotada por un atacante para comprometer el pipeline completo de entrenamiento o inferencia, incluso si el propio modelo y los datos usados son completamente seguros y confiables.

La seguridad del modelo y los datos es solo una parte de la superficie de ataque total del sistema; las dependencias de software (librerías, frameworks) que ejecutan el pipeline son otra parte real y separada que también puede tener vulnerabilidades explotables, y enfocarse exclusivamente en una parte mientras se ignora la otra deja una brecha de seguridad genuina sin que el equipo necesariamente se dé cuenta de esa exposición real.

Lección 24 — Seguridad de API keys y credenciales de modelos

Cualquier persona que encuentre esa clave expuesta podría usarla para hacer llamadas al modelo a costa de la cuenta del propietario legítimo, generando un costo económico no autorizado que la organización original tendría que pagar.

Durante el tiempo que la clave estuvo expuesta públicamente (aunque haya sido breve), bots automatizados que escanean constantemente repositorios públicos en busca de credenciales filtradas podrían haberla copiado y almacenado ya; eliminar el repositorio después no revierte ese hecho, por lo que la única protección real garantizada es revocar inmediatamente esa clave comprometida específica y generar una nueva.

Lección 25 — Aislamiento y sandboxing de cargas de trabajo de inferencia

Limita el impacto potencial si el proceso de inferencia fuera comprometido de alguna forma (por ejemplo, mediante una vulnerabilidad en el código de procesamiento), evitando que ese compromiso se extienda hacia otros sistemas o datos fuera de ese entorno aislado específico.

Si el proceso de inferencia tuviera acceso amplio a otros sistemas críticos de la red, un atacante que lograra comprometer ese proceso específico podría usar ese punto de entrada para moverse lateralmente hacia sistemas más sensibles; con permisos restringidos y aislamiento técnico real, ese mismo compromiso quedaría contenido dentro de ese entorno específico, limitando significativamente el daño máximo potencial que el atacante podría causar.

Módulo 6: Operación y respuesta

Lección 26 — Red teaming de sistemas de IA

Intenta activamente atacar y manipular el sistema (probando prompt injection, jailbreaks, y otras técnicas conocidas) desde la perspectiva de un adversario real, para descubrir vulnerabilidades explotables antes de que un atacante malicioso real las encuentre primero.

Si el equipo de red teaming interno descubre la vulnerabilidad antes del lanzamiento, hay una oportunidad real de corregirla sin que haya ocurrido ningún daño real todavía; si en cambio un atacante malicioso la descubre primero después de que el sistema ya está en producción, esa explotación ya habría causado algún tipo de daño real (financiero, de reputación, de datos) antes de que el equipo siquiera se enterara del problema.

Lección 27 — Detección de abuso y uso malicioso en producción

Un volumen inusualmente alto de consultas en poco tiempo, patrones repetitivos de intentos de manipulación con variaciones sutiles (sugiriendo prueba sistemática de jailbreak), o consultas que buscan sistemáticamente extraer información sensible del sistema.

Un usuario legítimo típicamente hace consultas variadas y espontáneas según sus necesidades reales cambiantes, mientras alguien probando sistemáticamente pequeñas variaciones del mismo tipo de solicitud (cambiando ligeramente la redacción cada vez) sugiere un intento deliberado de encontrar una formulación específica que logre evadir una restricción, un patrón de comportamiento notablemente distinto y más sospechoso que el uso natural y diverso de un usuario común.

Lección 28 — Respuesta a incidentes de seguridad específicos de IA

Analizar la técnica específica de jailbreak usada para entender exactamente cómo evadió las defensas existentes, y actualizar los guardrails o el filtrado del sistema específicamente para prevenir esa misma técnica u otras similares en el futuro.

Bloquear solo esa conversación específica trata el síntoma inmediato, pero no corrige la vulnerabilidad subyacente real que permitió que esa técnica de manipulación funcionara en primer lugar; sin analizar exactamente cómo esa técnica evadió las defensas existentes y actualizar los guardrails en consecuencia, cualquier otro atacante que use esa misma técnica (u otro usuario que la descubra de forma independiente) podría repetir exitosamente el mismo ataque en el futuro.

Lección 29 — Cumplimiento normativo de seguridad de IA (EU AI Act, marcos regulatorios)

Obligaciones adicionales de transparencia, evaluación de riesgos, supervisión humana, y documentación rigurosa del sistema, significativamente más estrictas que las que aplicarían a un sistema de IA de bajo riesgo o de uso general.

Una decisión incorrecta o injusta sobre un préstamo bancario puede tener consecuencias financieras significativas y reales en la vida de una persona, mientras una mala recomendación musical tiene consecuencias mínimas; los marcos regulatorios como el EU AI Act clasifican el nivel de riesgo (y por tanto el rigor de los requisitos exigidos) precisamente en función de esas consecuencias reales potenciales de una decisión incorrecta del sistema.

Lección 30 — Proyecto final: postura de seguridad de IA completa lista para producción

Modelado de amenazas específico + defensa en profundidad (filtrado de entrada/salida, guardrails) + seguridad de la cadena de suministro de modelos + protecciones de privacidad + red teaming previo al lanzamiento + plan de respuesta a incidentes específico de IA.

Intenta mencionar al menos dos pilares: modelado de amenazas, defensa en profundidad con guardrails, seguridad de la cadena de suministro o red teaming con plan de respuesta a incidentes.